OOPs in BusBookingSystem

1. Abstracttion

Used via Interfaces:

// Application/Common/Interfaces/IAppDbContext.cs

```
public interface IAppDbContext
{
    DbSet<User> Users { get; }
    DbSet<Booking> Bookings { get; }
    Task<int> SaveChangesAsync(CancellationToken cancellationToken);
    // ...
}
```

// Application/Common/Interfaces/IJwtTokenGenerator.cs

```
public interface IJwtTokenGenerator
{
    string GenerateToken(User user);
}
```

// Application/Common/Interfaces/IEmailService.cs

```
public interface IEmailService
{
    Task SendBookingConfirmationEmailAsync(...);
    Task SendBookingCancellationEmailAsync(...);
    Task SendEmailAsync(string toEmail, string subject, string htmlBody, ...);
```

}

Before (without abstraction)

- Handlers would directly new AppDbContext(...)
- Changing DB (e.g., from Postgres to SQL Server) would require touching every handler

After (without abstraction)

- Handlers depend on IAppDbContext — they never know the concrete type
- Only DependencyInjection.cs changes — handlers are untouched

Why it's better:

The Application layer has zero knowledge of EF Core, PostgreSQL, or SMTP. It only knows the contract.

2.Inheritance

What it is: A class inheriting state and behavior from a parent class.

BaseEntity — Abstract Base Class

// Domain/Common/BaseEntity.cs

```
public abstract class BaseEntity
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; set; }
}

// Every entity inherits it:
```

```
public class User : BaseEntity { ... }
public class Bus : BaseEntity { ... }
public class Booking : BaseEntity { ... }
public class BusOperator : BaseEntity { ... }
public class Trip : BaseEntity { ... }
public class Payment : BaseEntity { ... }
……
```

AppDbContext inherits DbContext

```
public class AppDbContext : DbContext, IAppDbContext
{
    public override Task<int> SaveChangesAsync(...)
    {
        // Auto-sets UpdatedAt on every save
        foreach (var entry in ChangeTracker.Entries<BaseEntity>())
            if (entry.State == EntityState.Modified)
                entry.Entity.UpdatedAt = DateTime.UtcNow;

        return base.SaveChangesAsync(cancellationToken);
    }
}
```

Controllers inherit ControllerBase

```
public class BookingsController : ControllerBase { ... }
public class AdminController : ControllerBase { ... }
public class AuthController : ControllerBase { ... }
```

Before (without BaseEntity)

- Every entity manually declares Id, CreatedAt, UpdatedAt — 12+ times
- UpdatedAt logic scattered or forgotten in each save
- Inconsistent ID generation (some Guid.NewGuid(), some not)

After (with BaseEntity)

- Declared once in BaseEntity, inherited by all
- AppDbContext.SaveChangesAsync() handles it centrally for all entities
- Guaranteed consistent across all entities

Why it's better: Single source of truth for cross-cutting entity concerns. Adding a new audit field (e.g., CreatedBy) means changing one class, not twelve.

3. Encapsulation:

What it is: Bundling data with the methods that operate on it, and controlling access.

// Bookings/DTOs/BookingDto.cs

```
public class BookingDto
{
    public Guid Id { get; set; }
    public string BusName { get; set; } = string.Empty;
    public decimal TotalAmount { get; set; }
    public string Status { get; set; } = string.Empty;
    // ... only what the API consumer needs
}
```

```
// The domain Booking entity is NEVER exposed directly
public class Booking : BaseEntity
{
    public string? CancellationReason { get; set; }
    public User Customer { get; set; } = null!;  // navigation — never sent to client
    // ...
}


public class CreateBookingCommandHandler :
IRequestHandler<CreateBookingCommand, BookingConfirmationDto>
{
    private readonly IAppDbContext _context;       // private, readonly
    private readonly IEmailService _emailService;  // private, readonly


    public CreateBookingCommandHandler(IAppDbContext context, IEmailService
emailService)
    {
        _context = context;
        _emailService = emailService;
    }
}
```

Before (no encapsulation):

- Domain entities returned directly from controllers
- Navigation properties like Customer.PasswordHash could leak to clients
- Dependencies mutable after construction

After (with encapsulation):

- DTOs act as a protective layer — internal model changes don't break the API
- Only explicitly mapped fields appear in the response
- readonly fields prevent accidental reassignment

Why it's better: The PasswordHash field on User never accidentally ends up in a JSON response. The API contract is stable even when the domain model evolves.

4. Polymorphism

What it is: One interface, many implementations — the same call behaves differently depending on the concrete type.

CQRS Handlers — all implement IRequestHandler<T, R>

// Same interface, completely different behavior:

public class LoginCommandHandler : IRequestHandler<LoginCommand, AuthResponseDto> { ... }

public class CreateBookingCommandHandler : IRequestHandler<CreateBookingCommand, BookingConfirmationDto> { ... }

public class ApproveOperatorCommandHandler : IRequestHandler<ApproveOperatorCommand, bool> { ... }

public class GetUserBookingsQueryHandler : IRequestHandler<GetUserBookingsQuery, List<BookingDto>> { ... }

// AuthController — same _mediator.Send() call, different handler invoked each time

await _mediator.Send(new LoginCommand { ... });

await _mediator.Send(new RegisterCustomerCommand { ... });

```
await _mediator.Send(new RegisterOperatorCommand { ... });
```

MediatR dispatches polymorphically:

```
// AuthController — same _mediator.Send() call, different handler invoked each time
await _mediator.Send(new LoginCommand { ... });
await _mediator.Send(new RegisterCustomerCommand { ... });
await _mediator.Send(new RegisterOperatorCommand { ... });
```

ProcessLogin — method reuse across three login types:

// AuthController.cs

```
[HttpPost("admin-login")]
public async Task<ActionResult<AuthResponseDto>> AdminLogin([FromBody]
LoginCommand command)
{
    command.Role = UserRole.Admin;
    return await ProcessLogin(command);  // same method
}

[HttpPost("user-login")]
public async Task<ActionResult<AuthResponseDto>> UserLogin([FromBody]
LoginCommand command)
{
    command.Role = UserRole.Customer;
    return await ProcessLogin(command);  // same method
}
```

private async Task<ActionResult<AuthResponseDto>> ProcessLogin(LoginCommand command) { ... }

Before (no polymorphism):

- Controller has a giant if/switch to decide which service to call
- Adding a new command requires modifying the controller
- Three login endpoints with duplicated try/catch logic

After (with polymorphism):

- _mediator.Send() resolves the right handler automatically
- New command = new handler class, zero changes to existing code (Open/Closed Principle)
- ProcessLogin private method handles all three uniformly

Why it's better: You can add 20 more commands without touching a single controller. MediatR's dispatch is the polymorphism engine here.

5. Missing Custom Exception Hierarchy (No Polymorphic Exception Handling)

Current Code

// CreateBookingCommand.cs

throw new Exception("Trip not found or not available");

throw new Exception("Bus not available for booking");

throw new Exception("Seats already booked: ...");

throw new Exception("Either BusId or TripId must be provided");

// CancelBookingCommand.cs

throw new Exception("Booking not found");

```
throw new Exception("Booking is already cancelled");
throw new Exception("Only confirmed bookings can be cancelled");

// RegisterCustomerCommand.cs
throw new Exception("Email already in use.");

// LoginCommand.cs
throw new UnauthorizedAccessException("Invalid credentials.");
```

Everything is either a raw Exception or UnauthorizedAccessException. The controller catches Exception and returns BadRequest for everything — a 404 (not found) and a 409 (conflict) both return 400.

What OOP Requires: Exception Inheritance Hierarchy

```
// Domain/Common/Exceptions/DomainException.cs — AFTER
public abstract class DomainException : Exception
{
    protected DomainException(string message) : base(message) { }
}

// Domain/Common/Exceptions/NotFoundException.cs — AFTER
public class NotFoundException : DomainException
{
    public NotFoundException(string entity, object id)
        : base($"{entity} with id '{id}' was not found.") { }
}
```

// Domain/Common/Exceptions/ConflictException.cs — AFTER

```
public class ConflictException : DomainException
{
   public ConflictException(string message) : base(message) { }
}
```

// Domain/Common/Exceptions/BusinessRuleException.cs — AFTER

```
public class BusinessRuleException : DomainException
{
   public BusinessRuleException(string message) : base(message) { }
}
```

6. Controllers (or a global middleware) can now handle them polymorphically:

// API/Middleware/ExceptionHandlingMiddleware.cs — AFTER

```
catch (NotFoundException ex)    => Results.NotFound(ex.Message);       // 404
catch (ConflictException ex)    => Results.Conflict(ex.Message);       // 409
catch (BusinessRuleException ex)=> Results.BadRequest(ex.Message);     // 400
catch (UnauthorizedAccessException ex) => Results.Unauthorized();      // 401
```

Before :

- throw new Exception("Booking not found") → controller returns 400 throw new NotFoundException("Booking", id) → middleware returns 400
- Every controller has its own try/catch block
- Can't distinguish "not found" from "conflict" from "business rule"
- 5 controllers × 3 actions = 15 duplicate try/catch blocks

After:

- throw new NotFoundException("Booking", id) → middleware returns 404
- One global middleware handles all exceptions
- Each exception type maps to the correct HTTP status code
- Zero try/catch in controllers

7. Notification Entity Doesn't Inherit BaseEntity (Broken Inheritance)

Current Code:

// Domain/Entities/Notification.cs

```
public class Notification   // ← NO BaseEntity inheritance!
{
   public Guid Id { get; set; }           // manually declared
   public DateTime CreatedAt { get; set; } // manually declared
   // No UpdatedAt
}
```

→Every other entity inherits BaseEntity. Notification manually re-declares Id and CreatedAt, and skips UpdatedAt entirely. This breaks the consistency the inheritance hierarchy was designed to provide.

What OOP Requires: Consistent Inheritance

// Domain/Entities/Notification.cs — AFTER

```
public class Notification : BaseEntity  // ← inherit like everyone else
{
   public Guid UserId { get; set; }
```

```
    public string Title { get; set; } = string.Empty;
    public string Message { get; set; } = string.Empty;
    public NotificationType Type { get; set; }  // ← enum, not string
    public bool IsRead { get; set; } = false;
    public Guid? RelatedBookingId { get; set; }
    // Id, CreatedAt, UpdatedAt come from BaseEntity
}
```

Before:

- Notification manually declares Id and CreatedAt
- AppDbContext.SaveChangesAsync() auto-sets UpdatedAt for all BaseEntity entries — but Notification is skipped
- Inconsistency — 12 entities use BaseEntity, 1 doesn't

After:

- Inherited from BaseEntity — no duplication
- Notification now gets UpdatedAt auto-managed too
- All 13 entities consistent